STAGE 1 : TOKENIZER

Input: Apple is a fruit

Tokenize: ["apple", "is", "a", "fruit"]

Convert to IDs: [15, 22, 8, 90]

Output: token_ids

STAGE 2 : EMBEDDINGS

Every token becomes a vector.

Example:

apple -> [0.2, 0.8, 0.4]

is   -> [0.1, 0.5, 0.7]

Embedding Matrix: Vocab Size × Embedding Dimension (10000 × 512)

Example: embedding_matrix[15]

Returns: [0.2,0.8,0.4...]

STAGE 3 : POSITIONAL EMBEDDINGS

Transformer doesn't know order.

Apple eats dog

Dog eats apple

Without position: same tokens

So add position information.

Position Matrix: 512 × 512

Example:

final_embedding =

token_embedding +

Position_embedding;

STAGE 4 : SELF ATTENTION

This is the brain.

Input: The cat drank milk because it was hungry.

Question: What is "it" ?

Attention learns:

it → cat

Step 1

Create

Q

K

V

Matrices.

Q = XWq

K = XWk

$V = XW_v$

Step 2

Similarity

$Q \times K^T$

Example:

cat -> 0.9

milk -> 0.2

hungry -> 0.7

Step 3

Softmax

Attention =

$\text{softmax}(QK^T / \sqrt{dk})$

Step 4

Output

Attention @ V

STAGE 5 : MULTI HEAD ATTENTION

Instead of one attention:

Head1 -> Grammar

Head2 -> Context

Head3 -> Subject

Head4 -> Object

Example: 8 heads

Output: Concat(heads)

Then: Linear Layer

STAGE 6 : ADD & NORM

Transformer Block

Attention+Skip Connection+LayerNorm

x = x + attention

x = layernorm(x)

STAGE 7 : FEED FORWARD NETWORK

Every token independently passes through:

Linear

|

ReLU

|

Linear

W1 = Expand knowledge

W2 = Compress knowledge

or

W1 = Learn features

W2 = Combine features

Formula: FFN(x)=W2(ReLU(W1x+b1))+b2

Example:

512

↓

2048

↓

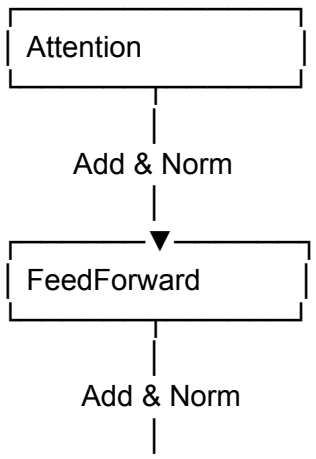
512

STAGE 8 : TRANSFORMER BLOCK

One encoder block:

Input
↓
Attention
↓
Add & Norm
↓
Feed Forward
↓
Add & Norm

Architecture:



Output

STAGE 9 : STACK BLOCKS

Real LLM:

Block 1
Block 2
Block 3
...
Block N

STAGE 10 : PRETRAINING

This is where the model learns language.

BERT (Masked LM)

Input: Apple is a [MASK]

Target: fruit

STAGE 11 : LOSS

Model predicts: [0.01,0.02,0.80,0.17]

Target: fruit

Cross Entropy:

loss = CrossEntropy(pred,target)

STAGE 12 : BACKPROPAGATION

Calculate gradients: dLoss/dW

Update: W = W - lr*gradient

Optimizer: AdamW

STAGE 13 : TEXT GENERATION

After training:

Input: Apple is a

Model: fruit

Next: that

Next: grows

Next: on

Next: trees

Loop:

```
while not stop:
  predict_next_token()
  append_token()
```

Text

↓
Tokenizer
↓
Token IDs
↓
Embedding
↓
Position Embedding
↓
Transformer Block 1
↓
Transformer Block 2
↓
Transformer Block 3
↓
...
↓
Transformer Block N
↓
Linear Layer
↓
Softmax
↓
Next Token Prediction
↓
Generated Text